Lecture 9 · Wed 10/28/2026

CS160 Compilers — Fall 2026

Type Checking.

Yu Feng

Where the checker sits.



semantic analysis

the pass after parsing that asks what the program *means*, not how it is written. The first pass that can reject a program it could read perfectly well. We call it *the checker*.

Monday ended with `print(f(1, 2))` for a one-parameter `f` compiling and printing 1. The parser cannot see anything wrong with it, and clang cannot either. Today is the pass that can. First, why such a pass is worth having.

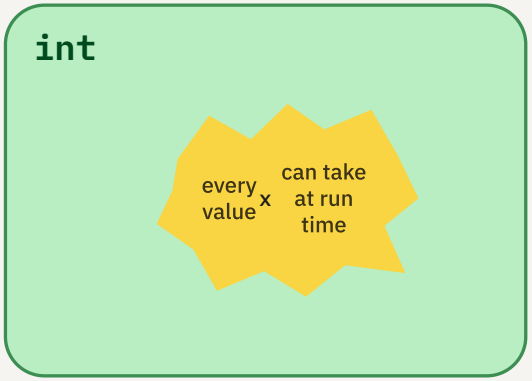
We would like to prove programs correct. We cannot.

Programs crash, or compute the wrong thing. We would like to *prove*, before running, that a program has no error.

But no algorithm can do that for every program: for any interesting property, there is no checker that is right about all programs. This is a theorem (Rice, 1953), not a limitation of today's tools.

So: give up? No. **Check something simpler than the program.**

Check an abstraction of the program instead.



concrete value

what the program actually computes when it runs: `10`, `-3`, `True`

abstract value

a description that stands for many concrete values at once, and throws the rest away

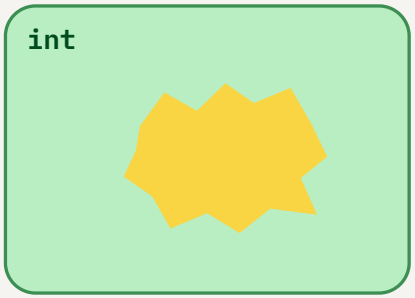
abstraction

replacing the program's concrete values by abstract ones, and reasoning about those.

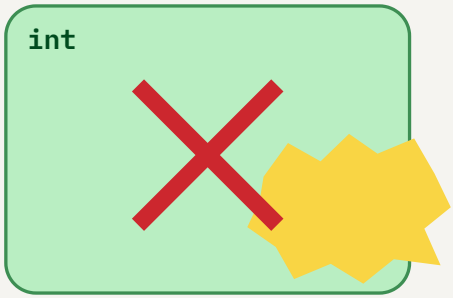
```
x: int = 10          concrete:  x is 10          abstract:  x is an int
y: int = x + 1       concrete:  y is 11          abstract:  y is an int
```

We cannot know what every program computes. We *can* know, for every program, what *kind* of thing each expression computes. That kind is its **type**, and types are the abstraction this course uses.

The abstraction must be sound.



sound: yellow inside green



not sound: yellow leaves green

sound

the abstract value always *contains* the concrete value: whatever the program really computes falls inside what the type says. `int` is sound for `10`; `bool` would not be

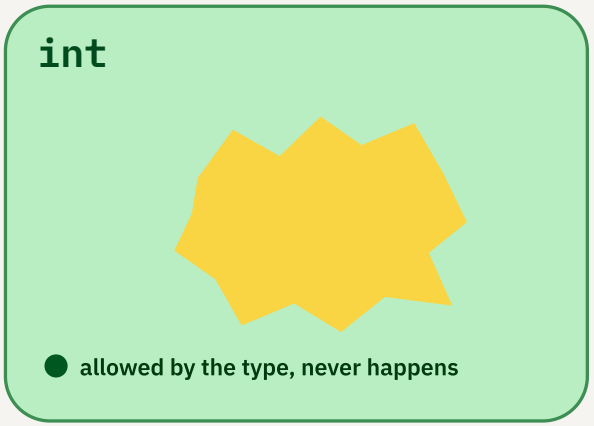
overapproximate

contain, possibly with room to spare — the type may allow more than the program will ever do, never less.

Why it matters: if the abstraction is sound and reports *no error*, the real program has no error of that kind either. The checker says `x + 1` is an `int`; because the abstraction is sound, running it produces an `int` — never a crash from adding a `bool` to a number.

A type checker is exactly this: run the program on types instead of values, and reject any program where the types do not fit together.

What the abstraction costs.



false positive

a program the checker rejects that would in fact have run without error.

falsepos.py · verified, rejected by typecheck.py

```
b: bool = True
x: int = 0
if False:
    x = b + 1        # never runs — but the checker rejects it:
                     line 4: expected int, found bool
```

The type says b could be any `bool`, and `bool + 1` has no rule, so the line is rejected although it never executes. That is the price of checking the abstraction instead of the program: some fine programs are refused. The trade is worth it: every program it accepts is guaranteed free of type errors.

Where the types come from.

type checking

the programmer writes the types (`x: int`, `def f(a: int) -> int`) and the checker verifies that every use is consistent with them. C, Java, ChocoPy

type inference

the programmer writes none; the compiler works out the best type of every expression and complains if they cannot be made to fit. ML, OCaml, Haskell.

ChocoPy checks: every variable and every parameter carries a declared type, so the checker never has to guess — it only has to *verify*. That is what the rest of today builds.

What the checker knows about a program.

run.py · verified: prints True · this program stays with us to the end of the lecture

```
x: int = 3                          # line 1
def inc(a: int) -> int:             # line 2
    return a + 1                    # line 3
print(inc(x) < 5)                   # line 4
```

Γ_top = {x: int, inc: (int) → int} lines 1, 4

Γ' = {a: int} line 3 ↑ then look here

what the checker knows, read off the program

```
Γ_top = { x: int,  inc: (int) → int }      R = ⊥      lines 1 and 4 are checked here
Γ'    = { a: int },  then Γ_top            R = int    line 3 (inc's body) is checked here
```

symbol table
the record of every declared name and what it stands for — in Lecture 3 a *place*, today a *type*. The manual calls it 0; we write Γ (gamma)

Γ_top
the table for the *top level*: names declared outside every function — here x and `inc`. Lecture 8's `globals`. 'Top' means the outermost part of the program, not the top of a stack

scope (Monday's word)
one table per region of the program. `inc`'s body gets its own table Γ', with Γ_top behind it; looking a name up tries Γ' first, then Γ_top

R
the return type the enclosing function promised — line 2's `-> int` — so that line 3's `return` can be checked against it. ⊥ at top level: no enclosing function.

Line 2 tells the checker that `inc` takes an `int` and returns an `int`; inside `inc`, a is known and `return` must give an `int`. What every name is, and what the current function must return — the rules need nothing else.

The types of ChocoPy.

```
int      bool                    the two types every rule today mentions
(ChocoPy has more — strings, lists — Monday and PA4.  No rule changes shape.)
```

Two types cover `run.py` and every rule today: a variable's declared type, an expression's type, a parameter's and a result's type are all drawn from these.

Monday adds the rest — strings, lists — and the two literals they bring (`[]`, `None`); no rule changes shape.

A judgment, and the shape of a rule.

```
Γ ⊢ e : T          for an expression:  in environment Γ (slide 8), expression e has type T
Γ, R ⊢ s           for a statement:  s is allowed, given what return must produce (R, slide 8)

O, M, C, R ⊢ e : T   how the manual (§5.2) writes it.  Its O is our Γ.  M and C are for
                     classes, which we do not have: strike them.  Its R rides on every
                     judgment; ours appears only where a statement needs it (slide 13).
```

$$\frac{\text{premises}}{\text{conclusion}}\ [\text{name}]$$

judgment
a claim that is true or false: *in environment Γ, e has type T*

typing rule
a bar: *if* the **premises** above it hold, *then* the **conclusion** below it holds. A rule with no premises is an **axiom**

⊢ · :
⊢ reads *proves*; : reads *has type*

side condition
a fact written beside a rule — looked up, not derived.

These rules are the definition of ChocoPy's types; nothing else says what the language means. Slide 5's ‘run the program on types’ means: apply these rules to the tree.

Rules for expressions: literals, variables, operators.

```
-------------- [int]        ------------------ [bool-true]      ------------------- [bool-false]
Γ ⊢ n : int                  Γ ⊢ True : bool                     Γ ⊢ False : bool

Γ(id) = T, where T is not a function type          Γ ⊢ e1 : int    Γ ⊢ e2 : int
---------------------------------------- [var-read]  --------------------------- [arith]
Γ ⊢ id : T                                            Γ ⊢ e1 op e2 : int
                                                         op ∈ {+, -, *, //, %}

Γ ⊢ e : int               Γ ⊢ e : bool              Γ ⊢ e1 : bool   Γ ⊢ e2 : bool
---------------- [negate]  ----------------- [not]  ------------------------------- [and], [or]
Γ ⊢ -e : int              Γ ⊢ not e : bool          Γ ⊢ e1 and e2 : bool
```

`Γ(id) = T`
look `id` up in Γ. If it is not there, `[var-read]` cannot fire — *undeclared name*

∈
is one of.

Read `[arith]` aloud: *if e1 is an int and e2 is an int, then e1 op e2 is an int.* Slide 5's rule for `x + 1`, written down. `[not]`, `[and]`, `[or]`: `bool` in, `bool` out — `if 1 and x:` is not ChocoPy.

Rules for expressions: comparison and function call.

```
Γ ⊢ e1 : int    Γ ⊢ e2 : int                Γ ⊢ e1 : bool    Γ ⊢ e2 : bool
--------------------------- [int-compare]   ------------------------------ [bool-compare]
Γ ⊢ e1 ./ e2 : bool                         Γ ⊢ e1 ./ e2 : bool
   ./ ∈ {<, <=, >, >=, ==, !=}                 ./ ∈ {==, !=}

Γ(f) = (T1, ..., Tn) → T0        Γ ⊢ e1 : T1   ...   Γ ⊢ en : Tn
------------------------------------------------------------- [invoke]
Γ ⊢ f(e1, ..., en) : T0
```

`./`
whichever comparison operator was written

two compare rules
`==` on two `int`s is `[int-compare]`; on two `bool`s, `[bool-compare]`; nothing types `1 == True`

function type `(T1, ..., Tn) → T0`
parameter types, then the result type — what Γ holds for a function name.

`[invoke]`: exactly *n* argument premises, one per parameter, each of the parameter's type. Monday's `f(1, 2)` dies here — `f` has one parameter and the call has two arguments.

Rules for statements.

A statement has no type. Its judgment is `Γ, R ⊢ s` (slide 10) — *statement s is well-formed in Γ* — and a declaration also hands back the new environment, written `=> Γ[x ↦ T]`.

```
Γ ⊢ e : T   x ∉ dom(Γ)                          Γ(x) = T   Γ ⊢ e : T
------------------------------ [var-init]        ---------------------- [var-assign-stmt]
Γ, R ⊢ x: T = e   => Γ[x ↦ T]                    Γ, R ⊢ x = e

Γ ⊢ c : bool   Γ, R ⊢ s1 … sn   Γ, R ⊢ t1 … tm      Γ ⊢ c : bool   Γ, R ⊢ s1 … sn
------------------------------------ [if-elif-else]  ---------------------------- [while]
Γ, R ⊢ if c: s1 … sn else: t1 … tm                    Γ, R ⊢ while c: s1 … sn

Γ ⊢ e : R                    Γ ⊢ e : T               Γ ⊢ e : T   T ∈ {int, bool}
----------------- [return-e]  ------------ [expr-stmt]  ------------------------- [print]
Γ, R ⊢ return e               Γ, R ⊢ e                 Γ, R ⊢ print(e)
```

well-formed
obeys the rules; the word for a statement, since it has no type to report

`=>`
and afterwards the environment is: the one rule that changes Γ

`[print]`
ours, not the manual's, and ours for the whole quarter: `print` is predefined and takes one `int` or `bool` (`str`, when strings arrive). Monday shows the manual's encoding once

`x ∉ dom(Γ)`
`x` is not already declared here

`Γ[x ↦ T]`
Γ plus one binding: after the declaration, `x` has type `T`

`⊥`
`R` at top level: no enclosing function, so `return` has nothing to return to.

One discipline runs through the storing rules — initialize, assign, return: the value's type must *equal* the declared one. Monday meets the two literals (`[]`, `None`) that take their type from the place they land. `[if-elif-else]`, `[while]`: the condition must be a `bool`. `[print]`: slide 17 runs it on `run.py`'s line 4.

Functions.

```
Γ(f) = (T1, ..., Tn) → T        Γ' = Γ ⊕ {x1: T1, ..., xn: Tn}        Γ', T ⊢ s1 … sk
------------------------------------------------------------------------------------ [func-def]
Γ, ⊥ ⊢ def f(x1: T1, ..., xn: Tn) -> T: s1 … sk
```

⊕

Γ with these bindings added — slide 13's `Γ[x ↦ T]` for all the parameters at once

`Γ'`

a *new* environment: the parameters are visible in the body and nowhere else

`Γ', T ⊢`

the body is checked with `R = T`: inside `f`, every `return e` must produce exactly a `T`.

Two conditions the manual states in words rather than as rules. *Return completeness* (§2.8.5): a function returning `int`, `str` or `bool` must `return` a value on every path — a walk over the statement list, where a `while` never counts.

Where does `Γ(f)` *come from?* Pass 1 reads every `def` header into `Γ_top`; pass 2 checks every body and every top-level statement. Without pass 1, a function could not call itself.

A program, checked · 1/4

Back to slide 8's program: pass 1.

run.py · slide 8's program, shown again

```
x: int = 3                          # line 1
def inc(a: int) -> int:             # line 2
    return a + 1                    # line 3
print(inc(x) < 5)                   # line 4
```

```
pass 1, the headers:    Γ_top = { x: int,  inc: (int) → int }      R = ⊥
pass 2, the body:       check line 1, then the def (lines 2–3), then line 4, each under Γ_top
```

Slide 8's table, now built by pass 1. Three statements to check. Each one is a derivation whose bottom line is `Γ_top, ⊥ ⊢ statement`; the next three slides are those derivations. Monday runs this same program.

A program, checked · 2/4

Line 1 and the function.

line 1, by [var-init]

```
---------- [int]
Γ ⊢ 3 : int        x ∉ dom(Γ)
------------------------------ [var-init]
Γ, ⊥ ⊢ x: int = 3    => Γ[x ↦ int]
```

lines 2–3, by [func-def] · Γ' = Γ ⊕ {a: int}, body checked with R = int

```
Γ'(a) = int
------------ [var-read]   ------------ [int]
Γ' ⊢ a : int              Γ' ⊢ 1 : int
---------------------------------------- [arith]
Γ' ⊢ a + 1 : int       (and int is this body's R)
------------------------------------------------ [return-e]
Γ', int ⊢ return a + 1
------------------------------------------------------------------------ [func-def]
Γ, ⊥ ⊢ def inc(a: int) -> int: return a + 1
```

derivation
the finished stack of bars: a proof that the bottom judgment holds, each step justified by a named rule

well-typed
a derivation exists.

Read each one from the bottom up: the statement, the rule whose conclusion has its shape, the smaller obligations that rule leaves, down to axioms and lookups. The function's body is checked in `Γ'`, where `a` is known, with `R = int` so the `return` has something to match — and only the `return` line carries an `R`: the expressions above it need just `Γ'`.

A program, checked · 3/4

Line 4, as one tree.

Γ = Γ_top, with x now bound

```
                        Γ(x) = int
                        ------------- [var-read]
  Γ(inc) = (int) → int   Γ ⊢ x : int
  ---------------------------------- [invoke]   ----------- [int]
  Γ ⊢ inc(x) : int                               Γ ⊢ 5 : int
  ------------------------------------------------------------- [int-compare]
  Γ ⊢ inc(x) < 5 : bool          bool ∈ {int, bool}
  ------------------------------------------------------------- [print]
  Γ, ⊥ ⊢ print(inc(x) < 5)
```

Five bars, one per node of the AST. The root — the only statement, so the only judgment carrying R — is `[print]`; the leaves are a lookup and a literal. Every expression in the program now has a type written on it — `inc(x)` is `int`, the comparison is `bool` — and codegen will read those types instead of working them out again (Monday).

A program, checked · 4/4

The same program with one line broken.

run_bad.py

```
x: int = 3
def inc(a: int) -> int:
    return a + 1
print(inc(x) < True)              # line 4:  5 became True
```

```
Γ ⊢ inc(x) : int          Γ ⊢ True : bool
-----------------------------------------  [int-compare] needs True : int.    Have bool.  STUCK.
Γ ⊢ inc(x) < True : ???                    [bool-compare] needs inc(x) : bool.  Have int.  STUCK.
```

ill-typed

no derivation exists: at some node, no rule's premises can be satisfied.

Both comparison rules get stuck on the same node, so there is no tree and the program is rejected: `line 4: cannot compare int with bool` (verified). The stuck node knows its line number; that is the whole error message.

Monday; PA3.

Monday: first the checker's code — one `case` per rule — then lists, and the two literals with no type of their own (`[]`, `None`): the one idea PA4 needs for them; then the yellow half of slide 5's picture: the rules for what a program *does* when it runs, in the same bar-and-premise shape, on this same program.

PA3 (due Fri 10/30): functions plus this checker; report the *first* error's line number. The rules and their names are the manual's §5, with slide 13's two departures: `[print]`, and ‘the same type’ where the manual writes $\leq_a$.

